🧠 GymemoGame: Comprehensive Source Code Documentation

Project Name: GymemoGame (Cognitive Adventure Platform)
Date Generated: 2026-03-15

🏛️ ส่วนที่ 1: สถาปัตยกรรมระบบหลังบ้าน (Backend Architecture)

1.1 จุดเริ่มต้นของโปรเจกต์ (Entry Point)

File: `backend/src/server.js`
เป็นส่วนสำคัญของ API ทำหน้าที่จัดการการเชื่อมต่อ ความปลอดภัย และการกำหนดเส้นทาง (Routing)

```
require('dotenv').config()
const express = require('express')
const cors = require('cors')
const connectDB = require('./config/db')
const authRoutes = require('./routes/authRoutes')
const scoreRoutes = require('./routes/scoreRoutes')
const progressionRoutes = require('./routes/progressionRoutes')
const analysisRoutes = require('./routes/analysisRoutes')
const dailyRoutes = require('./routes/dailyRoutes')
const { errorHandler } = require('./middleware/errorMiddleware')

const app = express()

const createAdmin = require('./scripts/createAdmin')

// Initialize Database connection and seed admin
connectDB().then(() => {
  createAdmin()
})

// Setup Cron Jobs for daily task resets
const setupCronJobs = require('./services/cronService')
setupCronJobs()

// Global Middlewares
app.use(cors())
app.use(express.json())

// Rate Limiting to prevent API abuse
const { apiLimiter } = require('./middleware/rateLimiter')
app.use('/api', apiLimiter)

// Health Check API
app.get('/', (req, res) => {
  res.json({ message: '🧠 Gymemo API Server', version: '1.0.0' })
```

```
})

// Main API Routes
const adminRoutes = require('./routes/adminRoutes')
app.use('/api/auth', authRoutes)
app.use('/api/scores', scoreRoutes)
app.use('/api/progression', progressionRoutes)
app.use('/api/analysis', analysisRoutes)
app.use('/api/daily', dailyRoutes)
app.use('/api/admin', adminRoutes)

// Error Handler
app.use(errorHandler)

const PORT = process.env.PORT || 3001
app.listen(PORT, () => {
  console.log(`🚀 Server is running on port ${PORT}`)
})
```

1.2 การออกแบบโมเดลข้อมูลผู้ใช้งาน (User Data Modeling)

File: `backend/src/models/User.js`
การกำหนดโครงสร้างข้อมูล (Schema) สำหรับระบบพิสูจน์ตัวตนและโปรไฟล์ผู้เล่น

```
const mongoose = require('mongoose')
const bcrypt = require('bcryptjs')

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please enter a name'],
    maxlength: [50, 'Name cannot exceed 50 characters']
  },
  phone: {
    type: String,
    required: [true, 'Please enter a phone number'],
    unique: true,
    trim: true,
  },
  password: {
    type: String,
    required: [true, 'Please enter a password'],
    minlength: [6, 'Password must be at least 6 characters'],
    select: false
  },
  highScore: { type: Number, default: 0 },
  totalKeys: { type: Number, default: 0 },
```

```
    avatar: { type: String, default: 'avatar-1' },
    role: { type: String, enum: ['user', 'admin'], default: 'user' }
  }, {
    timestamps: true
  })

  // Encryption middleware for password security
  userSchema.pre('save', async function (next) {
    if (!this.isModified('password')) {
      next()
    }
    const salt = await bcrypt.genSalt(10)
    this.password = await bcrypt.hash(this.password, salt)
  })

  userSchema.methods.comparePassword = async function (enteredPassword) {
    return await bcrypt.compare(enteredPassword, this.password)
  }

  module.exports = mongoose.model('User', userSchema)
```

1.3 โมเดลการวิเคราะห์เกม (Game Analysis Model)

File: `backend/src/models/GameAnalysis.js`
โครงสร้างข้อมูลสำหรับวัดผลความสามารถของสมองในด้านต่างๆ

```
const mongoose = require('mongoose')

const gameAnalysisSchema = new mongoose.Schema({
    guestId: { type: String, required: true, index: true },
    gameType: {
        type: String,
        required: true,
        enum: ['management', 'calculation', 'spatial', 'reaction']
    },
    level: { type: Number, required: true },
    subLevelId: { type: Number },

    // Core Metrics
    score: { type: Number, required: true },
    timeTaken: { type: Number },
    accuracy: { type: Number },
    moves: { type: Number },

    // Specific Domain Breakdown
    domainMetrics: {
        executive: { type: Number },
```

```
        memory: { type: Number },
        attention: { type: Number },
        calculation: { type: Number }
    },

    date: { type: Date, default: Date.now, index: true }
}, {
    timestamps: true
})

module.exports = mongoose.model('GameAnalysis', gameAnalysisSchema)
```

1.4 ตรรกะทางธุรกิจ: ส่วนจัดการสำหรับผู้ดูแลระบบ (Admin Controller)

ไฟล์: `backend/src/routes/adminRoutes.js`
รวบรวมตรรกะสำหรับการเข้าถึงข้อมูลของผู้ดูแลระบบและการตรวจสอบข้อมูล

```
const express = require('express')
const router = express.Router()
const { protect, admin } = require('../middleware/authMiddleware')
const User = require('../models/User')
const GameAnalysis = require('../models/GameAnalysis')

// Retrieve all user accounts
router.get('/users', protect, admin, async (req, res) => {
    try {
        const users = await User.find({}).select('-password')
        res.json({ success: true, count: users.length, data: users })
    } catch (error) {
        res.status(500).json({ success: false, message: error.message })
    }
})

// System-wide statistics for management reporting
router.get('/stats', protect, admin, async (req, res) => {
    try {
        const [totalUsers, totalGuests, totalScores] = await Promise.all([
            User.countDocuments({}),
            PlayerProgress.countDocuments({}),
            GameAnalysis.countDocuments({})
        ])

        // Aggregated accuracy metrics
        const averageScores = await GameAnalysis.aggregate([
            {
                $group: {
                    _id: '$gameType',
```

```
                    avgScore: { $avg: '$score' },
                    count: { $sum: 1 }
                }
            }
        ])

        res.json({
            success: true,
            data: { totalUsers, totalGuests, totalScores, averageScores }
        })
    } catch (error) {
        res.status(500).json({ success: false, message: error.message })
    }
})

module.exports = router
```

🎨 ส่วนที่ 2: แอปพลิเคชันส่วนหน้า (Frontend Application)

2.1 ชั้นบริการ API (API Service Layer)

File: `frontend/src/lib/api.ts`
Handling all outgoing requests to the backend API services.

```
export const API_BASE_URL = process.env.NEXT_PUBLIC_API_BASE_URL ??
'http://localhost:3001/api'

export async function loginUser(phone: string, password: string) {
  const res = await fetch(`${API_BASE_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ phone, password }),
  })
  const data = await res.json()
  if (!res.ok) throw new Error(data.message || 'Failed to login')
  return data
}

export async function submitScore(token: string, score: number, moves: number,
timeTaken: number) {
  const res = await fetch(`${API_BASE_URL}/scores`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer
${token}` },
    body: JSON.stringify({ score, moves, timeTaken }),
  })
  return res.json()
```

```
}

export async function adminGetStats(token: string) {
  const res = await fetch(`${API_BASE_URL}/admin/stats`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer
${token}` },
  })
  return res.json()
}

export async function adminGetUsers(token: string) {
  const res = await fetch(`${API_BASE_URL}/admin/users`, {
    method: 'GET',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer
${token}` },
  })
  return res.json()
}
```

2.2 สถาปัตยกรรมสถานะส่วนกลาง (Authentication Context)

File: `frontend/src/contexts/AuthContext.tsx`
จัดการเซสชันของผู้เล่นและรักษาความปลอดภัยในการเข้าถึงแอปพลิเคชัน

```
'use client'
import React, { createContext, useContext, useState, useEffect } from 'react'
import { useRouter } from 'next/navigation'
import Cookies from 'js-cookie'
import { getUserProfile } from '@/lib/api'

export function AuthProvider({ children }: { children: React.ReactNode }) {
    const [user, setUser] = useState<any>(null)
    const [token, setToken] = useState<string | null>(null)
    const [loading, setLoading] = useState(true)
    const router = useRouter()

    useEffect(() => {
        const storedToken = Cookies.get('token')
        if (storedToken) {
            setToken(storedToken)
            fetchUser(storedToken)
        } else {
            setLoading(false)
        }
    }, [])
```

```
    const fetchUser = async (authToken: string) => {
        try {
            const data = await getUserProfile(authToken)
            if (data.success) { setUser(data.data) }
            else { logout() }
        } catch (error) { logout() } finally { setLoading(false) }
    }

    const login = async (authToken: string, userData: any) => {
        Cookies.set('token', authToken, { expires: 7 })
        setToken(authToken)
        setUser(userData)
        router.refresh()
    }

    const logout = () => {
        Cookies.remove('token')
        setToken(null)
        setUser(null)
        window.location.href = '/'
    }

    return (
        <AuthContext.Provider value={{ user, token, loading, login, logout }}>
            {children}
        </AuthContext.Provider>
    )
}
```

2.3 แผนที่โลกและตรรกะการดำเนินเกม (Progression Logic)

File: `frontend/src/lib/levelSystem.ts`
หัวใจหลักของระบบตรรกะสำหรับการดำเนินเนื้อเรื่องผ่านหมู่บ้านทั้ง 10 แห่ง

```
export const PLAYS_PER_VILLAGE = 12;
export const MAX_KEYS = 9;

export function recordPlay(
  p: GymemoProgressV2,
  villageId: number,
  scoreGained: number,
  gameType?: 'management' | 'calculation' | 'spatial' | 'reaction'
): GymemoProgressV2 {
  let nextP = { ...p, villages: { ...p.villages } }
  const key = String(villageId)
  const vp = { ...(nextP.villages[key] ?? { playsCompleted: 0, expTubeFilled:
false }) }
```

```
  // Logic to track how many games are played in each village
  const newPlays = Math.min(vp.playsCompleted + 1, PLAYS_PER_VILLAGE * 99)
  const tubeFilled = newPlays >= PLAYS_PER_VILLAGE

  if (gameType) {
    nextP.totalScore += scoreGained;
  }

  vp.playsCompleted = newPlays;
  vp.expTubeFilled = tubeFilled;
  nextP.villages[key] = vp;

  // Handling Village Unlocking workflow
  if (tubeFilled && villageId < 10) {
    if (!nextP.unlockedVillages.includes(villageId + 1)) {
        nextP.unlockedVillages = [...nextP.unlockedVillages, villageId + 1]
    }
  }

  return nextP
}

export function resetGameProgress(p: GymemoProgressV2): GymemoProgressV2 {
  // Returns user to Level 1 while maintaining phone identity and guest ID
  return {
    ...p,
    villages: {},
    unlockedVillages: [1],
    totalScore: 0,
    introSeen: false
  }
}
```

2.4 การจัดการแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard)

File: `frontend/src/app/admin/page.tsx`
ส่วนแสดงผล Dashboard สำหรับการตรวจสอบข้อมูลผู้เล่นแบบ Real-time และการส่งออกข้อมูล

```
'use client'
import { useState, useEffect } from 'react'
import { useRouter } from 'next/navigation'
import { useAuth } from '@/contexts/AuthContext'
import { adminGetUsers, adminGetStats, adminGetExportScores } from '@/lib/api'

export default function AdminDashboard() {
    const { user, token } = useAuth()
```

```
    const router = useRouter()
    const [stats, setStats] = useState<any>(null)
    const [users, setUsers] = useState<any[]>([])
    const [loading, setLoading] = useState(true)

    useEffect(() => {
        if (!token || (user && user.role !== 'admin')) {
            router.push('/')
            return
        }
        fetchData()
    }, [token, user])

    const fetchData = async () => {
        try {
            setLoading(true)
            const [s, u] = await Promise.all([
                adminGetStats(token!),
                adminGetUsers(token!)
            ])
            setStats(s.data)
            setUsers(u.data)
        } catch (err) {
            console.error('Data sync failed:', err)
        } finally {
            setLoading(false)
        }
    }

    const exportToCSV = (data: any[], filename: string) => {
        if (data.length === 0) return
        const headers = Object.keys(data[0])
        const csvContent = [
            headers.join(','),
            ...data.map(row => headers.map(h => row[h]).join(','))
        ].join('\n')

        const blob = new Blob([csvContent], { type: 'text/csv' })
        const url = window.URL.createObjectURL(blob)
        const a = document.createElement('a')
        a.href = url
        a.download = `${filename}.csv`
        a.click()
    }

    return (
        <div className="admin-container p-16">
            <h1 className="text-4xl font-black mb-8">Admin Portal Control 👑</h1>

            {stats && (
                <div className="grid grid-cols-3 gap-12 mb-16">
                    <div className="bg-white p-12 rounded-3xl shadow-xl">
                        <p className="text-slate-400 font-bold">REGISTERED
USERS</p>
```

```
                        <h2 className="text-6xl font-black">{stats.totalUsers}
</h2>
                    </div>
                    <div className="bg-white p-12 rounded-3xl shadow-xl">
                        <p className="text-slate-400 font-bold">TOTAL SCORE
LOGS</p>
                        <h2 className="text-6xl font-black">{stats.totalScores}
</h2>
                    </div>
                    <div className="bg-white p-12 rounded-3xl shadow-xl">
                        <p className="text-slate-400 font-bold">ACTIVE GUESTS</p>
                        <h2 className="text-6xl font-black">{stats.totalGuests}
</h2>
                    </div>
                </div>
            )}

            <div className="bg-white p-12 rounded-[3rem] shadow-2xl">
                <h3 className="text-2xl font-black mb-8">Registered Player
Directory</h3>
                <table className="w-full border-collapse">
                    <thead>
                        <tr className="border-b-2 border-slate-100 text-slate-400
uppercase text-xs">
                            <th className="p-4 text-left">Name</th>
                            <th className="p-4 text-left">Phone Identifier</th>
                            <th className="p-4 text-right">Progress</th>
                        </tr>
                    </thead>
                    <tbody>
                        {users.map(u => (
                            <tr key={u._id} className="border-b border-slate-50
hover:bg-slate-50">
                                <td className="p-4 font-bold">{u.name}</td>
                                <td className="p-4 text-slate-500">{u.phone}</td>
                                <td className="p-4 text-right font-black text-
indigo-600">
                                    {u.highScore?.toLocaleString()} pts
                                </td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            </div>
        </div>
    )
}
```